



Universidad Nacional de Colombia

Facultad de ciencias

Departamento de matemáticas

Administración cuantitativa de
riesgos financieros

TimeGAN para el cálculo del VaR mediante datos sintéticos

Estudiante:

Jose Miguel Acuña Hernandez
jacunah@unal.edu.co

Profesor:

Oscar Javier Lopez Alfonso
ojlopeza@unal.edu.co

Repositorio:

TimeGAN-VaR

Contenido

1. Introducción	1
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Marco teórico	2
4. Metodología	2
5. Implementación en Python	2
5.1. Estructura del Proyecto	2
5.2. Conformación del portafolio	3
5.3. Entrenamiento de la red neuronal	4
5.4. Metodo de Montecarlo para el VaR usando los datos sintéticos de TimeGAN	6
5.5. Cálculo del VaR con metodos convencionales	6
6. Resultados	6
7. Conclusiones	6
8. Bibliografía	6

1. Introducción

El Valor en Riesgo (VaR) se ha establecido como una métrica fundamental en la gestión de riesgos financieros, permitiendo cuantificar la máxima pérdida esperada para un portafolio en un horizonte temporal específico y con un nivel de confianza determinado. Los métodos tradicionales para su cálculo incluyen el enfoque paramétrico, la simulación histórica y la simulación de Monte Carlo, cada uno con sus propias limitaciones, especialmente evidentes durante períodos de alta volatilidad.

Las Redes Neuronales Recurrentes (RNN) han demostrado una notable capacidad para modelar series temporales financieras gracias a su arquitectura que incorpora conexiones cíclicas, permitiéndoles capturar dependencias temporales complejas en datos secuenciales. Paralelamente, las Redes Generativas Adversarias (GAN) han revolucionado el aprendizaje automático con su enfoque competitivo entre generador y discriminador, mostrando un potencial significativo para la generación de datos sintéticos realistas.

TimeGAN combina ambas tecnologías, integrando un componente supervisado que obliga al generador a preservar explícitamente la dinámica temporal de los datos originales. Esta innovación resulta particularmente relevante para aplicaciones financieras, donde capturar correctamente las estructuras de dependencia temporal es crucial.

Este proyecto implementa TimeGAN para generar series temporales sintéticas de retornos financieros y las utiliza para calcular el VaR mediante simulación de Monte Carlo. Nuestra metodología comprende el preprocesamiento de datos históricos, el entrenamiento del modelo TimeGAN, la generación de trayectorias sintéticas y la comparación de las

estimaciones de VaR con métodos convencionales, buscando determinar si este enfoque proporciona mediciones de riesgo más robustas, en comparación con los métodos tradicionales.

2. Objetivos

2.1. Objetivo general

Implementar y evaluar la arquitectura TimeGAN como metodología alternativa para el cálculo del Valor en Riesgo (VaR) de un portafolio diversificado compuesto por acciones, bonos gubernamentales, ETFs y oro, determinando su eficacia frente a métodos convencionales de estimación.

2.2. Objetivos específicos

1. Diseñar una arquitectura de proyecto modular y reproducible en GitHub, siguiendo principios de código limpio y documentación rigurosa para facilitar su replicabilidad.
2. Configurar un entorno de desarrollo basado en conda con integración de Cursor AI, Python y TensorFlow 2.0 con Keras que garantice la reproducibilidad computacional del proyecto.
3. Adaptar e implementar la arquitectura TimeGAN tomando como base el repositorio original desarrollado por gusxo (<https://github.com/gusxo/TimeGAN-keras>), que ofrece una implementación eficiente en Keras de la arquitectura propuesta por Yoon et al. (2019). Esta implementación será integrada dentro de un flujo de trabajo completo para el cálculo dinámico del VaR, permitiendo la modificación paramétrica del portafolio analizado según las necesidades del usuario.
4. Calcular el VaR del mismo portafolio utilizando metodologías convencionales (paramétrica, simulación histórica y Monte Carlo tradicional) como base comparativa.
5. Evaluar comparativamente la precisión y robustez de las diferentes metodologías mediante métricas cuantitativas adecuadas, con especial énfasis en su comportamiento ante escenarios de alta volatilidad y eventos extremos.

3. Marco teórico

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue. Ut in risus volutpat libero pharetra tempor. Cras vestibulum bibendum augue. Praesent egestas leo in pede. Praesent blandit odio eu enim. Pellentesque sed dui ut augue blandit sodales. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam nibh. Mauris ac mauris sed pede pellentesque fermentum. Maecenas adipiscing ante non diam sodales hendrerit.

4. Metodología

A

5. Implementación en Python

5.1. Estructura del Proyecto

La estructura del repositorio se organiza de la siguiente manera:

Estructura de carpetas

```

TimeGAN-VaR/
├── data/
│   ├── input/ # Datos originales para conformar el portafolio
│   ├── processed/ # Retornos logarítmicos procesados del portafolio
│   └── output/ # Resultados y métricas exportadas del análisis
├── images/ # Visualizaciones y diagramas generados
├── models/ # Modelos TimeGAN entrenados (.h5)
├── notebooks/
│   └── implementacion.ipynb # Notebook principal
├── src/ # Código fuente modular del proyecto
│   ├── data/
│   │   ├── __init__.py
│   │   ├── loader.py # Carga y validación de datos financieros
│   │   ├── transform.py # Transformaciones para series temporales
│   │   └── windowing.py # Creación de ventanas para entrenamiento
│   ├── models/
│   │   ├── __init__.py
│   │   ├── timegan.py # Arquitectura TimeGAN completa
│   │   └── var_model.py # Cálculo de VaR con distintas metodologías
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── evaluation.py # Métricas de evaluación y validación
│   │   ├── visualization.py # Generación de gráficos y visualizaciones
│   │   └── __init__.py # Inicialización del paquete principal
├── tex/
│   └── proyecto_final.tex
├── .gitignore # Archivos y directorios excluidos del repositorio
├── environment.yml # Especificación del entorno Conda
├── LICENCE # Licencia MIT del proyecto
└── README.md # Documentación principal

```

5.2. Conformación del portafolio

La función `build_multi_asset_portfolio`, implementada en el módulo `src/data/transform.py`, permite construir un portafolio financiero diversificado a partir de una combinación de activos, asignando a cada uno un peso específico. El procedimiento es el siguiente:

- **Validación de entradas:** Se verifica que la cantidad de tickers coincida con la cantidad de pesos y que la suma de los pesos sea exactamente 1.
- **Descarga de datos:** Para cada ticker proporcionado (por ejemplo, acciones, bonos, ETFs, oro), se descargan los precios históricos usando la librería `yfinance`.
- **Alineación temporal:** Se determina la fecha de inicio común más reciente entre todos los activos, de modo que el portafolio solo considere el periodo en el que existen datos para todos los instrumentos.
- **Cálculo del portafolio:** Se calcula el valor del portafolio en cada fecha como la suma ponderada de los precios de cierre de cada activo, usando los pesos especificados.
- **Salida:** La función retorna un `DataFrame` con las columnas `Date` y `Close`, representando la evolución temporal del portafolio.

A continuación se muestra un ejemplo de cómo llamar esta función en Python para construir un portafolio con cuatro activos y pesos personalizados:

```

from src.data.transform import build_multi_asset_portfolio

# Definir los tickers y los pesos del portafolio
tickers = ("^GSPC", "AGG", "DBC", "GLD") # Portafolio de acciones, bonos, ETFs y oro
weights = (0.5, 0.3, 0.1, 0.1) # Pesos iguales para cada activo

# Construir el portafolio
portfolio_df = build_multi_asset_portfolio(tickers, weights)

# Visualizar las primeras filas
sns.set_style("whitegrid")
plt.figure(figsize=(10, 5))
sns.lineplot(x="Date", y="Close", data=portfolio_df, color="seagreen", linewidth=2)
plt.title("Evolución del valor del portafolio")
plt.xlabel("Fecha")
plt.ylabel("Valor de cierre ponderado")
plt.tight_layout()
plt.show()

```



5.3. Entrenamiento de la red neuronal

Para entrenar la arquitectura TimeGAN, primero es necesario transformar la serie temporal del portafolio en ventanas de tamaño fijo, lo que permite que la red neuronal aprenda patrones temporales de longitud consistente. En este proyecto, utilizamos ventanas de 250 días, que corresponden aproximadamente a un año de datos bursátiles.

La creación de estas ventanas se realiza mediante la función `create_windows` del módulo `src/data/windowing.py`. Esta función toma como entrada el DataFrame de retornos (o precios) y lo divide en subconjuntos secuenciales de longitud especificada (`window_size`), permitiendo además definir el `stride` (paso entre ventanas) y la columna objetivo si se requiere.

Previo a la creación de ventanas, los datos del portafolio se transforman y normalizan utilizando funciones del módulo `src/data/transform.py`, como `calculate_returns` para obtener los retornos logarítmicos y `normalize_data` para escalar los valores y facilitar el entrenamiento de la red.

A continuación se muestra un ejemplo de cómo se utilizan estas funciones en el notebook principal:

```

from src.data.transform import calculate_returns, normalize_data
from src.data.windowing import create_windows

# Calcular retornos logarítmicos

```

```
returns_df = calculate_returns(portfolio_df, method='log')

# Normalizar los datos
returns_norm, norm_params = normalize_data(returns_df, features=['Close'])

# Crear ventanas de 250 días con stride 1
windows = create_windows(returns_norm, window_size=250, stride=1)
```

El modelo TimeGAN, implementado en `src/models/timegan.py`, combina una red generativa adversaria (GAN) con componentes supervisados y autoencoders para preservar la dinámica temporal de las series. La arquitectura incluye los siguientes módulos principales:

- **Embedder y Recovery:** Codifican y decodifican las series temporales originales a un espacio latente.
- **Generator y Supervisor:** El generador produce secuencias sintéticas en el espacio latente, mientras que el supervisor ayuda a capturar dependencias temporales a largo plazo.
- **Discriminator:** Distingue entre secuencias reales y sintéticas en el espacio latente.

Durante el entrenamiento, se utilizan varias funciones de pérdida (*loss functions*):

- **Pérdida de autoencoder (MSE):** Minimiza la diferencia entre la serie original y la reconstruida por el embedder y recovery.
- **Pérdida supervisada (MSE):** Obliga al supervisor a predecir correctamente los siguientes pasos en el espacio latente.
- **Pérdida adversaria (BCE):** El generador intenta engañar al discriminador, que a su vez trata de distinguir entre datos reales y sintéticos.
- **Pérdida total:** Es una combinación ponderada de las anteriores, siguiendo la metodología propuesta en el artículo original de TimeGAN.

La división entre entrenamiento y prueba (*train/test split*) se realiza dentro de la función `timegan_train` del módulo `timegan.py`, donde se reserva un porcentaje de las ventanas para validación (por defecto, 20%). El entrenamiento se realiza únicamente sobre el conjunto de entrenamiento, mientras que el conjunto de prueba se utiliza para monitorear el desempeño y evitar sobreajuste.

A continuación se muestra un ejemplo de cómo se llama la función de entrenamiento de TimeGAN en el notebook principal:

```
from src.models.timegan import timegan_init, timegan_train

# Inicializar la arquitectura de TimeGAN
timegan_tuple = timegan_init(
    time_series_len=250,      # Longitud de la ventana
    features=returns_norm.shape[-1], # Número de variables
    rnn_units=64,            # Unidades RNN por capa
    rnn_layers=6             # Número de capas RNN
)

# Entrenar el modelo TimeGAN
history = timegan_train(
    windows,                 # Ventanas de entrenamiento
    timegan_tuple,           # Arquitectura inicializada
    epochs=4000,             # Número de épocas
    batch_size=16,          # Tamaño de lote
    learning_rate=0.0005,    # Tasa de aprendizaje
    test_size=0.2            # Proporción de datos para test
)

# Exportar el modelo entrenado
syn_generator = timegan_export_generator(timegan_tuple)
```

5.4. Metodo de Montecarlo para el VaR usando los datos sintéticos de TimeGAN

Lo primero que realizamos es generar los datos sintéticos de la serie temporal del portafolio, para esto, usamos la función `generator_gen` del módulo `src/models/timegan.py`. Esta función toma como entrada el modelo TimeGAN entrenado y el número de muestras a generar. Luego, preparamos los datos sintéticos para poder evaluar que tan similar es el comportamiento de los datos sintéticos con el comportamiento del portafolio real mediante `prepare_data_for_evaluation` para luego calcular las metricas “Divergencia KL, Discriminative score” y “Predictive score”.

```
from src.models.timegan import generator_gen

# Generar 5000 muestras sintéticas
synthetic_data = generator_gen(syn_generator, num_samples=5000)

# Preparar los datos para evaluación
from src.utils.evaluation import prepare_data_for_evaluation, calculate_kl_divergence,
    discriminative_score, predictive_score

real_windows, synthetic_windows = prepare_data_for_evaluation(
    returns_norm['Close'].to_numpy(), # 0 la columna de retornos normalizados
    correspondiente
    synthetic_data,
    window_size=250,
    stride=50
)

# Calcular métricas de similitud
kl_div = calculate_kl_divergence(real_windows, synthetic_windows)
disc_score = discriminative_score(real_windows, synthetic_windows)
pred_score = predictive_score(real_windows, synthetic_windows)

print(f"Divergencia KL: {kl_div:.4f}")
print(f"Discriminative Score: {disc_score:.4f}")
print(f"Predictive Score (MSE): {pred_score:.4f}")
```

5.5. Cálculo del VaR con metodos convencionales

6. Resultados

A

7. Conclusiones

a

8. Bibliografía

- [1] J. Yoon, D. Jarrett y M. van der Schaar, “Time-series Generative Adversarial Networks,” en *Advances in Neural Information Processing Systems*, 2019, págs. 5508-5518.
- [2] J. Yoon, *TimeGAN: Time-series Generative Adversarial Networks*, <https://github.com/jsyoon0823/TimeGAN>, 2019.
- [3] gusxo, *TimeGAN-keras: Implementation of TimeGAN with Keras*, <https://github.com/gusxo/TimeGAN-keras>, 2021.